

# Dance4Life: Implementação de um Sistema Multiagente para a Promoção do Envelhecimento Ativo

Carlos da Mota Bergueira (pg11605@alunos.uminho.pt)  
Diego Jefferson Mendes Silva (pg59999@alunos.uminho.pt)  
Filipa Araújo Pereira (pg42201@alunos.uminho.pt)  
Rui Manuel Martins Marques Rodrigues (pg7942@alunos.uminho.pt)

Universidade do Minho, Braga, Portugal

**Abstract.** O presente relatório descreve a conceção, modelação e implementação do protótipo do sistema Dance4Life, um ecossistema distribuído baseado em Sistemas Multiagente (SMA) desenvolvido em Python com recurso à framework SPADE (Smart Python Agent Development Environment). O sistema visa promover o envelhecimento ativo, correlacionando dados de movimento físico com estímulos musicais e variáveis de contexto social, no âmbito do paradigma aging in place. A arquitetura implementada compõe seis agentes especializados - ApiAgent, SensorAgent, CoordinatorAgent, HARAgent, EnvironmentAgent e DatabaseAgent -, comunicando através do protocolo FIPA-ACL (performativas request, agree, inform e failure) e transmitindo objetos estruturados via pydantic e jsonpickle. A implementação valida a viabilidade técnica do paradigma SMA para a coordenação sequencial e assíncrona de pipelines de saúde geriátrica, desde a perceção sensorial até à persistência em base de dados. São discutidos os resultados obtidos, as limitações arquiteturais identificadas e propostas concretas de evolução futura, incluindo paralelismo com asyncio.gather e integração de um Directory Facilitator para descoberta dinâmica de serviços.

**Keywords:** Sistemas Multiagente · SPADE · FIPA-ACL · Envelhecimento Ativo · Reconhecimento de Atividade (HAR)

## 1 Introdução

### 1.1 Domínio e Motivação

O envelhecimento demográfico global constitui um dos principais desafios sociais e de saúde pública da atualidade. A estimativa de que, até 2050, a população com mais de 65 anos atinja 1,5 mil milhões de pessoas impõe uma mudança de paradigma: a transição de cuidados clínicos hospitalares para modelos de envelhecimento ativo e monitorização remota na própria residência. É neste contexto que surge o ecossistema concetual Dance4Life, proposto como domínio de aplicação deste trabalho prático. O Dance4Life visa promover o envelhecimento ativo

através da inferência implícita de preferências do utilizador, correlacionando o movimento físico com estímulos musicais e interações sociais. A diversidade de dados envolvidos (sensores inerciais, sinais vitais, contexto sonoro e perfis sociais) não pode ser gerida eficientemente por uma arquitetura monolítica. Os SMA emergem como o paradigma tecnológico ideal para este cenário. A sua natureza modular, distribuída e assíncrona permite que entidades computacionais autónomas - os agentes - colaborem para perceber o ambiente, interpretar dados ruidosos e tomar decisões em tempo real.

## 1.2 Objetivos

O objetivo central deste projeto consiste na conceção e implementação de um protótipo distribuído baseado no paradigma SMA, focado no domínio do envelhecimento ativo, desenvolvido em Python com a biblioteca SPADE. Os objetivos são:

- Conceção Arquitetural: Desenhar uma arquitetura distribuída e modular, apoiada por metodologias Agent UML.
- Implementação de Agentes Inteligentes: Definir os agentes ApiAgent, SensorAgent, CoordinatorAgent, HARAgent, EnvironmentAgent e DatabaseAgent, com os seus comportamentos (CyclicBehaviour e OneShotBehaviour).
- Padronização FIPA-ACL: Implementar o fluxo conversacional request agree inform/failure em todas as interações ponto-a-ponto.
- Transmissão de Dados Estruturados: Garantir que o intercâmbio de mensagens utiliza objetos pydantic serializados com jsonpickle, em vez de strings simples.
- Orquestração Assíncrona: Desenvolver mecanismos de coordenação no CoordinatorAgent para gerir o pipeline sequencial sem bloqueios na thread principal.

## 1.3 Estrutura do Relatório

O relatório está organizado do seguinte modo: a Secção 2 apresenta a conceção e modelação do sistema (topologia, agentes e protocolos); a Secção 3 detalha a implementação técnica e o fluxo de dados; a Secção 4 apresenta os resultados e a análise crítica; e a Secção 5 conclui com recomendações de trabalho futuro.

## 2 Conceção e Modelação do Sistema

A fase de conceção seguiu os princípios da metodologia de modelação baseada em agentes (Agent UML - AUML). O desenvolvimento de um ecossistema focado no envelhecimento ativo exige o processamento de dados heterogêneos em tempo real (fisiológicos, inerciais e ambientais). A arquitetura de software foi desenhada para isolar responsabilidades computacionais, mitigando a conexão entre a captura de dados físicos e as inferências cognitivas ou sociais.

## 2.1 Arquitetura do Sistema Multiagente (Topologia)

A arquitetura do sistema Dance4Life foi concebida segundo uma topologia distribuída e hierárquica, inspirada nas melhores práticas identificadas no estado da arte dos Sistemas Multiagente aplicados ao envelhecimento ativo. A heterogeneidade dos dados envolvidos - sinais inerciais, contexto social, preferências musicais e metadados ambientais - exige uma infraestrutura capaz de integrar múltiplas fontes de informação, processá-las de forma sequencial e garantir fiabilidade em cenários sensíveis, como aplicações geriátricas. Por este motivo, a arquitetura adotada combina um modelo edge-to-cloud com um mecanismo de coordenação centralizada, assegurando coerência, modularidade e extensibilidade. No núcleo da topologia encontra-se o *CoordinatorAgent*, responsável por gerir o pipeline de processamento e garantir que as dependências lógicas entre etapas são respeitadas. A existência deste agente central é justificada pela natureza sequencial do domínio: não é possível inferir contexto social antes de reconhecer a atividade física, nem persistir dados na base de dados antes de validar a integridade das fases anteriores. Assim, o *CoordinatorAgent* atua como uma máquina de estados distribuída, delegando tarefas aos agentes especialistas e consolidando os resultados num fluxo ordenado e atómicamente consistente. Esta abordagem reduz a ligação entre agentes, facilita a substituição de módulos (por exemplo, a troca do *HARAgent* por um modelo de Machine Learning real) e aumenta a robustez global do sistema - um requisito crítico em aplicações de saúde e bem estar sénior. A periferia do sistema é composta pelo *ApiAgent* e pelo *SensorAgent*, que formam a camada de entrada (edge). O *ApiAgent* simula a origem dos dados provenientes de dispositivos externos, como smartwatches ou sensores inerciais, articulando-os num formato estruturado e enviando-os para o *SensorAgent*. Este último desempenha o papel de gateway da camada física, validando a receção dos dados e encaminhando-os para o *CoordinatorAgent*. Esta separação entre aquisição e processamento permite isolar falhas de dispositivos, reduzir a complexidade da camada cognitiva e preparar o sistema para cenários multiutilizador. A camada de processamento especializado é composta por três agentes distintos:

- *HARAgent*, responsável pela interpretação dos sinais físicos e pela inferência da atividade;
- *EnvironmentAgent*, que contextualiza a atividade com variáveis ambientais e sociais;
- *DatabaseAgent*, que assegura a persistência atómica dos dados na cloud.

Cada um destes agentes opera de forma autónoma e independente, comunicando através de mensagens FIPA ACL e trocando objetos estruturados serializados com jsonpickle. Esta articulação permite que cada componente evolua de forma isolada, suportando futuras extensões como novos sensores, modelos cognitivos mais avançados ou mecanismos de recomendação social. A topologia adotada reflete, assim, um equilíbrio entre centralização funcional e distribuição computacional, alinhando-se com arquiteturas modernas como JADE, PANGAEA e THOMAS, que privilegiam a interoperabilidade, a escalabilidade e a resiliência.

O resultado é um ecossistema multiagente capaz de integrar dados heterogêneos, operar em tempo real e fornecer suporte inteligente ao envelhecimento ativo, garantindo simultaneamente segurança, consistência e extensibilidade.

**Table 1.** Topologia e comunicações do sistema Dance4Life.

| Agente           | Tipo / Papel                    | Communicates With                                      |
|------------------|---------------------------------|--------------------------------------------------------|
| ApiAgent         | Gateway / Trigger externo       | SensorAgent                                            |
| SensorAgent      | Reativo periferia               | ApiAgent, CoordinatorAgent                             |
| CoordinatorAgent | Orquestrador máquina de estados | SensorAgent, HARAgent, EnvironmentAgent, DatabaseAgent |
| HARAgent         | Cognitivo HAR / ML              | CoordinatorAgent                                       |
| EnvironmentAgent | Contextual social/ambiental     | CoordinatorAgent                                       |
| DatabaseAgent    | Persistência Firebase           | CoordinatorAgent                                       |

**\*\*DIAGRAMA\*\***

## 2.2 Definição dos Agentes

**Classes, Behaviours e Performativas** A arquitetura do Dance4Life assenta num conjunto de seis agentes especializados, cada um responsável por uma função distinta dentro do ecossistema distribuído. A modelação seguiu os princípios de Agent UML e as normas FIPA ACL, garantindo modularidade, isolamento funcional e interoperabilidade entre componentes. Cada agente herda de `spade.agent.Agent` e implementa comportamentos concorrentes através de `CyclicBehaviour` e `OneShotBehaviour`. A comunicação entre agentes é orientada a objetos, recorrendo a modelos Pydantic serializados com `jsonpickle`.

**ApiAgent Gateway / Interface Externa** O `ApiAgent` representa a fronteira entre o sistema multiagente e o mundo exterior. É responsável por simular a origem dos dados provenientes de dispositivos periféricos (ex.: smartwatch, sensores inerciais ou aplicações móveis). Este agente isola os dados brutos num objeto estruturado, gera um `conversation-id` único via `uuid4()` e inicia o protocolo FIPA Request enviando o pedido ao `SensorAgent`. Behaviour principal:

- `RequestBehaviourSendSensorActivityToSensor` (`OneShotBehaviour`) - envolve os dados, define ontologia e metadados, envia o pedido e aguarda confirmação (agree) antes de terminar a transação.

Este agente garante que toda a comunicação que entra no sistema é validada, rastreável e semanticamente consistente.

**SensorAgent Agente Reativo de Periferia** O `SensorAgent` atua como controlador lógico da camada física. É responsável por receber os dados enviados pelo `ApiAgent`, validar a ontologia e reencaminhar a informação para o `CoordinatorAgent`. Funciona como um intermediário entre a aquisição sensorial e o

processamento cognitivo, isolando falhas e garantindo que apenas dados válidos entrem no pipeline. Behaviours:

- ReceiveBehaviourSensorAgent (CyclicBehaviour) - escuta continuamente mensagens FIPA ACL, valida ontologia e responde com agree ou failure.
- RequestBehaviourSendToCoordinator (OneShotBehaviour) - criado dinamicamente para enviar os dados ao CoordinatorAgent sem bloquear o ciclo principal, utilizando asyncio.Future.

Este agente assegura que o sistema permanece responsivo mesmo sob carga elevada.

**CoordinatorAgent Agente Orquestrador** O CoordinatorAgent é o núcleo lógico do sistema. Não executa cálculos complexos; a sua inteligência reside na capacidade de gerir o pipeline sequencial e delegar tarefas aos agentes especialistas. Atua como uma máquina de estados distribuída, garantindo que cada fase ocorre na ordem correta (HAR Contexto Persistência). Behaviours:

- ReceiveBehaviourCoordinatorAgent (CyclicBehaviour) - recebe dados do SensorAgent e inicia o pipeline.
- RequestBehaviourRequestFromHAR (OneShotBehaviour) - delega a fase de reconhecimento de atividade.
- RequestBehaviourRequestFromEnvironment (OneShotBehaviour) - solicita contexto social/ambiental.
- RequestBehaviourRequestToDatabase (OneShotBehaviour) - envia os dados consolidados para persistência.

Cada fase utiliza asyncio.Future para suspender a execução sem bloquear o agente, permitindo que o Coordinator continue a receber outras mensagens.

**HARAgent Agente Cognitivo (Human Activity Recognition)** O HARAgent é responsável pela interpretação dos sinais físicos e pela inferência da atividade do utilizador. Estimula o comportamento de um modelo de Machine Learning, transformando dados inerciais em semântica comportamental (ex.: dançar, caminhar, parado). Behaviours:

- ReceiveBehaviourHARAgent (CyclicBehaviour) - recebe pedidos do CoordinatorAgent, processa os dados e devolve um objeto CalculatedActivityData (Pydantic), garantindo validação de tipos e integridade semântica.

Este agente representa a camada cognitiva do sistema, podendo futuramente integrar modelos reais de HAR baseados em CNN/LSTM.

**EnvironmentAgent Agente de Contexto Social** O EnvironmentAgent complementa a interpretação da atividade com variáveis externas, como música em reprodução, densidade de pessoas na sala ou métricas sociais derivadas de preferências do utilizador. Este agente permite que o sistema evolua de uma monitorização puramente física para uma abordagem holística de envelhecimento ativo. Behaviours:

- ReceiveBehaviourEnvironmentAgent (CyclicBehaviour) - recebe o resultado do HARAgent e produz um objeto EnvironmentData, incluindo listas de relacionamento social e metadados ambientais.

Este agente aproxima o Dance4Life de arquiteturas modernas que integram suporte social e cognitivo.

**DatabaseAgent Agente de Persistência e Recursos** O DatabaseAgent isola todas as operações de input/output e comunicação com a infraestrutura externa (Firebase Firestore). Esta separação garante que falhas de rede ou problemas de persistência não afetam a lógica cognitiva dos restantes agentes. Behaviours:

- ReceiveBehaviourDatabaseAgent (CyclicBehaviour) - recebe pedidos de escrita, valida a ontologia e executa a operação save\_activity, devolvendo inform ou failure.

Este agente assegura atomicidade, rastreabilidade e consistência dos dados armazenados.

**Table 2.** Síntese dos agentes, behaviours e tipos.

| Agente           | Behaviour Principal                                                | Tipo                |
|------------------|--------------------------------------------------------------------|---------------------|
| ApiAgent         | RequestBehaviourSendSensorActivityToSensor                         | OneShotBehaviour    |
| SensorAgent      | ReceiveBehaviourSensorAgent RequestBehaviourSendToCoordinator      | Cyclic + OneShot    |
| CoordinatorAgent | ReceiveBehaviourCoordinatorAgent RequestBehaviourRequestHAR,Env,DB | Cyclic + 3x OneShot |
| HARAgent         | ReceiveBehaviourHARAgent                                           | CyclicBehaviour     |
| EnvironmentAgent | ReceiveBehaviourEnvironmentAgent                                   | CyclicBehaviour     |
| DatabaseAgent    | ReceiveBehaviourDatabaseAgent                                      | CyclicBehaviour     |

### 2.3 Protocolos de Comunicação e Performativas FIPA-ACL

A comunicação entre agentes no Dance4Life segue as normas definidas pela FIPA ACL (Foundation for Intelligent Physical Agents Agent Communication Language), garantindo interoperabilidade, previsibilidade e rastreabilidade em todas as interações. Esta escolha alinha o sistema com plataformas consolidadas como JADE, THOMAS e TeleCARE, que demonstram a importância de protocolos formais em ecossistemas distribuídos aplicados à saúde e ao envelhecimento ativo. A adoção de FIPA ACL permite que cada mensagem trocada entre agentes seja semanticamente explícita, contendo metadados essenciais como performative, ontology, conversation-id e reply-with. Esta estruturação é relevante em domínios geriátricos, onde a ambiguidade comunicacional pode comprometer a integridade dos dados clínicos ou gerar interpretações incorretas de sinais fisiológicos.

**Modelo de Interação** FIPA Request Interaction Protocol: Todas as interações ponto a ponto no Dance4Life seguem o FIPA Request Interaction Protocol, composto por três fases fundamentais:

1. Iniciação (request): O agente emissor (por exemplo, o ApiAgent ou o CoordinatorAgent) inicia a interação enviando uma mensagem com performativa request. Esta mensagem inclui:
  - ontologia partilhada ("sensor\_activity")
  - conversation-id único (gerado via uuid4())
  - objeto estruturado serializado com jsonpickle
 A utilização de objetos Pydantic garante type safety e evita ambiguidades nas mensagens textuais.
2. Avaliação (agree ou failure): O agente recetor valida a ontologia e a estrutura da mensagem.
  - Se a ontologia for reconhecida, responde com agree, comprometendo-se a processar o pedido.
  - Se a ontologia for inválida ou a mensagem estiver mal formada, responde imediatamente com failure, evitando processamento desnecessário e preservando a integridade do pipeline.
 Este mecanismo reflete boas práticas observadas em arquiteturas como SAAA, que enfatizam segurança, validação e governança de dados.
3. Resolução (inform ou failure): Após o processamento assíncrono, o agente recetor envia:
  - inform, contendo o resultado estruturado (ex.: CalculatedActivityData, EnvironmentData), ou
  - failure, caso ocorra erro, timeout ou inconsistência.
 Esta fase encerra a conversação, permitindo ao emissor prosseguir para a etapa seguinte do pipeline.

**Gestão de Conversações e Isolamento de Sessões** Para garantir isolamento entre múltiplas interações concorrentes, cada ciclo de comunicação é identificado por um conversation-id único. Todos os agentes filtram mensagens recebidas com base neste identificador, assegurando que:

- respostas não se misturam entre pipelines distintos
- o CoordinatorAgent consegue gerir múltiplas sessões em paralelo
- o sistema permanece determinístico mesmo sob carga elevada

Este mecanismo é inspirado no modelo de conversation tracking utilizado em JADE e THOMAS.

**Troca de Dados Estruturados** Embora o protocolo XMPP utilizado pelo SPADE contenha apenas texto, o Dance4Life implementa um mecanismo de comunicação orientada a objetos:

- Pydantic define modelos de dados com validação automática;

- jsonpickle serializa e desserializa objetos complexos para JSON;
- os agentes acessam diretamente aos atributos dos objetos, evitando parsing manual.

Este paradigma reduz erros semânticos, aumenta a legibilidade e aproxima o sistema de arquiteturas modernas baseadas em ontologias formais.

**Table 3.** Performativas FIPA-ACL utilizadas no sistema.

| Performativa | Direção |         | Significado                                   | Quando enviada               |
|--------------|---------|---------|-----------------------------------------------|------------------------------|
| request      | Emissor | Recetor | Pedido de execução de tarefa                  | Início de qualquer interação |
| agree        | Recetor | Emissor | Ontologia reconhecida; processamento iniciado | Após validação da ontologia  |
| inform       | Recetor | Emissor | Resultado positivo com dados estruturados     | Após execução bem-sucedida   |
| failure      | Recetor | Emissor | Erro, ontologia inválida ou timeout           | Em qualquer ponto de falha   |

A implementação dos protocolos FIPA ACL confere ao Dance4Life um nível elevado de formalismo comunicacional, essencial para aplicações sensíveis como o envelhecimento ativo. A combinação de ontologias validadas, isolamento de conversações e troca de objetos estruturados garante robustez, escalabilidade e alinhamento com o estado da arte em Sistemas Multiagente.

### 3 Implementação e Funcionamento (Baseado em SPADE)

A implementação do sistema Dance4Life foi realizada integralmente em Python, recorrendo à framework SPADE (Smart Python Agent Development Environment), que fornece uma infraestrutura robusta para a criação de agentes assíncronos baseados em XMPP e compatíveis com os protocolos FIPA ACL. Esta secção descreve a transposição da arquitetura conceptual para código executável, destacando os mecanismos de concorrência, isolamento de conversações, troca de objetos estruturados e orquestração sequencial do pipeline. A escolha da SPADE justifica-se pela sua aderência nativa ao modelo de agentes proposto no estado da arte, nomeadamente pela utilização de comportamentos (Behaviours), suporte à comunicação FIPA ACL e integração transparente com o motor assíncrono `asyncio`. Estas características permitem construir um ecossistema distribuído, modular e responsivo, adequado às exigências do domínio do envelhecimento ativo, onde a fiabilidade e a rastreabilidade são essenciais.

#### 3.1 Fluxo de Dados Pipeline Sequencial Assíncrono

Embora os Sistemas Multiagente sejam inerentemente concorrentes, o domínio do Dance4Life exige um pipeline sequencial, devido às dependências lógicas entre as fases de processamento. A atividade física deve ser reconhecida antes de ser contextualizada, e ambos os resultados devem ser validados antes de serem persistidos na base de dados. Assim, o `CoordinatorAgent` implementa uma máquina de estados distribuída que executa três fases ordenadas:



- Interpretação (HARAgent)
- Contextualização (EnvironmentAgent)
- Persistência (DatabaseAgent)

Para evitar bloqueios no event loop do agente, cada fase utiliza objetos `asyncio.Future`, permitindo suspender a execução local enquanto se aguarda a resposta remota, sem comprometer a capacidade do agente de receber outras mensagens. Este mecanismo garante que o sistema permanece responsivo, mesmo em cenários com múltiplas conversações simultâneas, aproximando o comportamento do Dance4Life de arquiteturas como THOMAS e PANGAEA, que privilegiam escalabilidade e processamento distribuído.

**\*\*DIAGRAMA\*\***

### 3.2 Gestão de Conversações e Isolamento de Mensagens

Para garantir o correto isolamento de mensagens num ambiente de alta concorrência, cada conversa é identificada por um `conversation-id` único gerado com `uuid4()` pelo `ApiAgent`. Todos os agentes filtram as mensagens recebidas com base neste identificador:

```
# Geração de conversation-id único (ApiAgent)
self.conversation_id = str(uuid.uuid4())

# Filtragem de mensagens por conversation-id (todos os agentes)
if reply.get_metadata('conversation-id') != self.conversation_id:
    continue # ignora mensagens de outras conversas
```

Adicionalmente, cada `behaviour` de delegação utiliza `asyncio.Future` para suspender a execução sem bloquear a thread principal do agente:

```
# CoordinatorAgent -- delegação com Future
future_har = asyncio.get_running_loop().create_future()
behaviour_har = self.agent.RequestBehaviourRequestFromHAR(
    ontology='sensor_activity', data=data,
    receiver='har_agent@localhost',
    conversation_id=conv_id, future=future_har
)
self.agent.add_behaviour(behaviour_har)
result_ok_har = await future_har # suspende até resolução
```

### 3.3 Troca de Mensagens Baseada em Objetos (Pydantic + Jsonpickle)

A arquitetura implementada não utiliza a passagem de texto simples no corpo das mensagens (o parâmetro `body` das mensagens FIPA-ACL no SPADE). O envio de strings brutas introduz fragilidades na extração de dados (parsing) e

difícil a manutenção do código em ecossistemas heterogêneos. Para colmatar esta limitação arquitetural do protocolo XMPP (que apenas transporta cadeias de caracteres), o sistema tira partido de duas bibliotecas complementares para implementar um mecanismo avançado de transferência de objetos (Data Transfer Objects):

- Estruturação com Pydantic: Os dados processados pelos agentes especialistas são modelados através de classes que herdam de `pydantic.BaseModel`. O `HARAgent` analisa e devolve o objeto complexo `CalculatedActivityData` (que valida tipos de dados para atividade, interesse e vetores temporais minuciosos). Simultaneamente, o `EnvironmentAgent` lida com a classe `EnvironmentData`, englobando metadados como `musica_id` e matrizes socioculturais como a `matching_list`. Esta abordagem assegura `type safety` nativo.
- Serialização e Desserialização com `Jsonpickle`: No momento de envio do `request` ou `inform`, o emissor converte instantaneamente as instâncias das classes em representações JSON serializadas (`jsonpickle.encode(data)`). No lado do recetor, o corpo da mensagem XMPP é decodificado na sua forma original (`jsonpickle.decode(msg.body)`), permitindo o acesso imediato e orientado a objetos aos atributos das classes (ex: `data.utilizador_id`).

Este paradigma de comunicação orientada a objetos aumenta a robustez, a `type safety` e a legibilidade da infraestrutura, mitigando falhas na interpretação semântica da informação partilhada na rede. O diagrama de classes UML a seguir representa a estrutura de dados criada e o fluxo de serialização para o isolamento no corpo das mensagens SPADE:

**\*\*DIAGRAMA\*\***

### 3.4 Validação de Ontologias

Cada agente recetor mantém uma lista local de ontologias suportadas (`ontology_list`). Antes de iniciar qualquer processamento, o agente verifica se a ontologia da mensagem recebida está nessa lista. Caso contrário, responde imediatamente com `failure`, libertando o emissor e evitando processamento desnecessário:

```
ontology_list = ['sensor_activity']

if ontology not in ontology_list:
    failure = msg.make_reply()
    failure.set_metadata('performative', 'failure')
    failure.body = f"Ontology '{ontology}' não suportada"
    await self.send(failure)
    return
```

### 3.5 Estrutura de Ficheiros do Projeto

O projeto está organizado nos seguintes módulos principais:

**Table 4.** Estrutura de ficheiros do projeto.

| Ficheiro                     | Conteúdo / Responsabilidade                                               |
|------------------------------|---------------------------------------------------------------------------|
| api_agent.py                 | ApiAgent trigger externo, geração de conversation-id                      |
| sensor_agent.py              | SensorAgent gateway da camada física, reencaminhamento para o Coordenador |
| coordinator_agent.py         | CoordinatorAgent orquestrador do pipeline sequencial em 3 fases           |
| har_agent.py                 | HARAgent reconhecimento de atividade, resposta com CalculatedActivityData |
| environment_agent.py         | EnvironmentAgent contexto social/ambiental, resposta com EnvironmentData  |
| database_agent.py            | DatabaseAgent persistência atômica via Firebase (save_activity)           |
| base_agent.py                | BaseAgent utilitários partilhados (build_request, wait_for_reply)         |
| model/data_model.py          | Modelos pydantic: CalculatedActivityData, EnvironmentData                 |
| services/firebase_service.py | Integração com Firebase Firestore                                         |
| utilities/helper.py          | Funções auxiliares (e.g., generate_mock_matching_list)                    |

## 4 Resultados e Análise Crítica

A implementação do protótipo Dance4Life permitiu validar a arquitetura proposta e demonstrar a viabilidade técnica de um sistema multiagente distribuído para o domínio do envelhecimento ativo. Os testes realizados evidenciaram o correto funcionamento do pipeline sequencial, a robustez da comunicação FIPA ACL e a eficácia da troca de objetos estruturados entre agentes. No entanto, a análise crítica revela também limitações arquiteturais e operacionais que devem ser consideradas para evoluções futuras.

### 4.1 Sumário do Funcionamento e Testes

Durante as sessões de teste e execução, o comportamento emergente da rede de agentes confirmou a conformidade com as regras de negócio desenhadas em Agent UML. Verificaram-se os seguintes resultados práticos:

- Integração Conversacional FIPA-ACL: A adoção das performativas padronizadas revelou-se robusta. O mecanismo de handshake (agree antes do processamento pesado + inform no final) forneceu garantias de entrega e evitou situações de bloqueio silencioso. Sempre que foi injetada uma mensagem com ontologia desconhecida, o sistema respondeu instantaneamente com failure, evidenciando a correta validação de domínio.
- Transação de Dados Complexos: A utilização da triada pydantic + objetos nativos + jsonpickle funcionou com qualidade. A capacidade de envolver variáveis heterogêneas (listas de emparelhamento geradas pelo EnvironmentAgent, marcadores temporais do HARAgent) e transitá-las entre processos sem recorrer à manipulação de strings provou ser eficiente e isenta de perdas semânticas.
- Gestão de Sessão via Identificadores: A geração de conversation-id único pela ApiAgent (uuid4()) garantiu o correto isolamento de mensagens. O CoordinatorAgent geriu múltiplas interações filtrando apenas as respostas associadas ao contexto em execução.

- Rastreabilidade do Fluxo: Os logs de execução revelaram o fluxo completo e ordenado das performativas entre todos os agentes, confirmando a sequência: request agree (processamento) inform/failure em cada par de agentes.
- Integração com Firebase: O DatabaseAgent integrou com sucesso o serviço firebase\_service para persistência atômica dos dados de atividade processados, completando o ciclo de vida da informação desde a periferia até à cloud.

## 4.2 Análise Crítica Estrangulamento Sequencial

Apesar dos resultados positivos, a análise crítica revela limitações estruturais que devem ser abordadas para garantir escalabilidade, resiliência e extensibilidade do sistema.

1. Restrição sequencial no CoordinatorAgent A limitação mais significativa reside no facto de o pipeline ser rigidamente linear:

```
await future_har -> await future_env -> await
future_db
```

Isto implica que:

- o tempo total de resposta é a soma das três fases
- o CoordinatorAgent não pode processar múltiplos pipelines em paralelo
- o sistema não escala bem para múltiplos utilizadores ou múltiplos dispositivos

Embora adequado para um protótipo, este modelo não é sustentável em cenários reais de envelhecimento ativo, onde dezenas de eventos podem ocorrer simultaneamente.

2. Ausência de Paralelismo entre Agentes Especialistas O HARAgent e o EnvironmentAgent são independentes, mas o sistema não explora essa independência. Num cenário real, seria possível:
  - processar HAR e contexto em paralelo
  - aplicar asyncio.gather para reduzir latência
  - permitir que o Coordinator gere múltiplas conversações simultaneamente
3. Falta de Descoberta Dinâmica de Serviços Atualmente, todos os agentes estão rigidamente configurados. Isto contrasta com arquiteturas modernas como THOMAS, que utilizam um Directory Facilitator para:
  - registar agentes dinamicamente
  - descobrir serviços em tempo real
  - substituir agentes sem alterar o código do coordenador

A ausência deste mecanismo reduz a flexibilidade e dificulta a evolução do sistema.

4. Dependência de um Único Ponto de Falha O CoordinatorAgent é um single point of failure. Se falhar:
  - o pipeline colapsa;
  - nenhum agente consegue completar o ciclo de processamento;
  - o sistema perde resiliência.

Arquiteturas distribuídas modernas tendem a evitar este tipo de centralização.

5. Simulação de Dados em Vez de Modelos Reais Embora adequado para um protótipo, o HARAgent e o EnvironmentAgent utilizam lógica simulada. Para aplicações reais, seria necessário:
  - integrar modelos de HAR baseados em CNN/LSTM;
  - incorporar sensores reais;
  - utilizar dados fisiológicos e sociais autênticos.

O protótipo Dance4Life demonstra que um SMA baseado em SPADE é tecnicamente viável para o domínio do envelhecimento ativo. A arquitetura é modular, clara e extensível, e a implementação respeita rigorosamente as normas FIPA ACL. No entanto, a escalabilidade, a resiliência e a autonomia do sistema podem ser melhoradas através de paralelismo, descoberta dinâmica de serviços e substituição do coordenador central por mecanismos mais distribuídos.

## 5 Conclusões e Recomendações para Melhoria

### 5.1 Conclusões

O desenvolvimento do protótipo Dance4Life permitiu demonstrar a viabilidade técnica e conceptual de um SMA aplicado ao domínio do envelhecimento ativo. A arquitetura distribuída concebida - composta por agentes especializados, comunicação FIPA ACL e troca de objetos estruturados - revelou-se adequada para integrar dados heterogêneos, processá-los de forma sequencial e garantir rastreabilidade e coerência em todas as fases do pipeline. A implementação em SPADE validou a capacidade dos agentes para operar de forma assíncrona, modular e isolada, refletindo princípios observados em arquiteturas de referência como JADE, PANGAEA e THOMAS. O CoordinatorAgent demonstrou ser eficaz na formulação do fluxo de dados, assegurando que cada etapa (HAR, contexto social e persistência) ocorre de forma ordenada e consistente. A utilização de Pydantic e jsonpickle permitiu uma comunicação orientada a objetos robusta, evitando ambiguidades semânticas e facilitando a evolução futura do sistema. Do ponto de vista funcional, o protótipo cumpriu todos os requisitos definidos:

- o pipeline executou-se corretamente;
- as ontologias foram validadas;
- as conversações foram isoladas;
- os dados foram persistidos com sucesso;
- e o sistema manteve responsividade mesmo sob múltiplas interações.

Em síntese, o Dance4Life demonstra que SMA são uma abordagem promissora para apoiar o envelhecimento ativo, permitindo integrar monitorização física, contexto social e persistência de dados num ecossistema distribuído, modular e extensível.

## 5.2 Recomendações e Trabalho Futuro

**Paralelismo de Comportamentos e Otimização Assíncrona** Para mitigar a latência acumulada no CoordinatorAgent, o processamento sequencial deve evoluir para um paradigma concorrente. Utilizando as primitivas avançadas do `asyncio`, o Coordenador poderá instanciar e aguardar pelos Futures do HARA-gent e do EnvironmentAgent em simultâneo:

```
# Proposta: execução paralela com asyncio.gather
future_har = asyncio.get_running_loop().create_future()
future_env = asyncio.get_running_loop().create_future()

self.agent.add_behaviour(RequestBehaviourRequestFromHAR(
    ..., future=future_har))
self.agent.add_behaviour(
    RequestBehaviourRequestFromEnvironment(
    ..., future=future_env))

# Aguarda ambos em paralelo
result_ok_har, result_ok_env = await asyncio.gather(
    future_har, future_env)

# Apenas a fase de persistência mantém-se sequencial
if result_ok_har and result_ok_env:
    result_ok_db = await future_db
```

Esta abordagem reduziria o tempo de resposta do sistema quase a metade, dado que a abstração de contexto social não tem de esperar pelo cálculo de atividade física. Apenas a etapa final (chamada ao DatabaseAgent) se manteria estritamente sequencial.

**Adição de Novos Agentes e Descoberta Dinâmica** A infraestrutura atual recorre a endereços estáticos (hardcoded, e.g., `har_agent@localhost`). Em trabalhos futuros, o sistema deve implementar mecanismos de Serviço de Diretório (Directory Facilitator ou Service Facilitator), permitindo que os agentes se registem dinamicamente e que o Coordenador descubra o serviço adequado em tempo de execução. Além disso, a arquitetura ganharia robustez com a inclusão de dois novos agentes:

- Security Agent: Para auditoria de acessos e monitorização contínua da frequência cardíaca, com autonomia para interromper sessões de dança em caso de fadiga detetada.
- Feedback Agent: Responsável por interagir de volta com o utilizador de forma proativa (e.g., recomendar uma música com base nos dados guardados), fechando o ciclo de recomendação inteligente.

**Segurança de Dados e Edge Computing** Considerando a sensibilidade clínica e demográfica dos dados gerados, é necessário o reforço da privacidade em conformidade com o RGPD. O desenvolvimento futuro deve englobar:

- Edge Computing: Transição do processamento cognitivo (modelos de ML emulados no HAR) para o próprio dispositivo do utilizador, garantindo que apenas parâmetros anonimizados, e não sinais biométricos brutos, viajam pela rede XMPP.
- Encriptação end-to-end: A comunicação deverá incorporar encriptação em todos os behaviours de rede.
- Aprendizagem Colaborativa Distribuída: Implementação de Federated Learning, onde os agentes partilham apenas os parâmetros do modelo, preservando a privacidade dos dados brutos.

**Melhorias de Código Identificadas** Na análise ao código existente, foram ainda identificadas as seguintes melhorias pontuais de implementação:

- BaseAgent: O método `request_and_wait` encontra-se comentado. A sua reativação e utilização sistemática pelos agentes reduziria significativamente a duplicação de código de envio e receção de mensagens.
- SensorAgent: A verificação `if reply and reply.get_metadata("conversation-id") != self.conversation_id: continue` deveria preceder a verificação de `if not reply`, como já implementado no CoordinatorAgent.
- Deduplicação dos RequestBehaviour: Os três RequestBehaviour do CoordinatorAgent (HAR, Environment, Database) são quase idênticos. Poderiam ser refatorados num único behaviour parametrizável, reduzindo a duplicação de código.

O Dance4Life constitui uma prova de conceito sólida que demonstra o potencial dos Sistemas Multiagente para apoiar o envelhecimento ativo. A arquitetura é clara, modular e extensível, e a implementação em SPADE valida a capacidade dos agentes para operar de forma distribuída e assíncrona. As recomendações apresentadas fornecem um roteiro concreto para transformar o protótipo num sistema mais escalável, inteligente e alinhado com as necessidades reais da população sénior.

## References

1. Bugaychenko, D., Dzuba, A.: Musical Recommendations and Personalization in a Social Network. In: Proc. RecSys '13, pp. 273–280. ACM (2013)
2. Camarinha-Matos, L.M., Castolo, O., Rosas, J.: A Multi-agent Based Platform for Virtual Communities in Elderly Care. In: IEEE ETFA (2002/2003)
3. Crista, V., Martinho, D.V., Marreiros, G.: Chat4Elderly: A Multi-Agent System for Personalized Wellness Using Generative AI and Wearable Technology. In: Proc. AAMAS 2025
4. Esmaeili, A., Ghorrati, Z., Matson, E.T.: Distributed Agent-Based Collaborative Learning in Cross-Individual Wearable Sensor-Based Human Activity Recognition. arXiv:2311.04236 (2023)
5. Ferri-Molla, I., Linares-Pellicer, J., et al.: Multi-Agent AI System for Adaptive Cognitive Training in Elderly Care. In: Proc. ICAART 2025

6. Fraile, J.A., Bajo, J., Corchado, J.M.: Multi-Agent Architecture for Dependent Environments. *Inteligencia Artificial* **13**(41) (2009)
7. Fraile Nieto, J.A., et al.: The THOMAS Architecture: A Case Study in Home Care Scenarios. Springer (2010)
8. Humayun, M., Jhanjhi, N.Z., et al.: Agent-Based Medical Health Monitoring System. *Sensors* **22**(8), 2820 (2022)
9. Ivaşcu, T., Negru, V.: Activity-Aware Vital Sign Monitoring Based on a Multi-Agent Architecture. *Sensors* **21**(12), 4181 (2021)
10. Mendes, S., Queiroz, J., Leitão, P.: Data Driven Multi-agent m-Health System to Characterize the Daily Activities of Elderly People. In: Proc. CISTI 2017
11. Özkara, M., Turan, M.: Personalized News Recommendation System. *Journal of Technology and Applied Sciences* (2022)
12. Paraschiv, E.A., Vevera, A.V., et al.: Towards Trustworthy AI Agents in Geriatric Medicine. *Future Internet* **18**, 75 (2026)
13. Sánchez San Blas, H., Mendes, A.S., et al.: A Multi-agent System for Data Fusion Techniques Applied to the IoT Enabling Physical Rehabilitation Monitoring. *Applied Sciences* **11**(1), 331 (2021)
14. Shakshuki, E., Reid, M.: Multi-Agent System Applications in Healthcare. *Procedia Computer Science* **52**, 252–261 (2015)
15. Teixeira, E., Fonseca, H., et al.: Wearable Devices for Physical Activity and Healthcare Monitoring in Elderly People. *Geriatrics* **6**(2), 38 (2021)
16. Vargemidis, D., Gerling, K., et al.: Wearable Physical Activity-Tracking Systems for Older Adults. *ACM Trans. Comput. Healthcare* **1**(1) (2020)
17. Vemuri, A., Decker, K., et al.: Multi agent architecture for automated health coaching. *Journal of Medical Systems* **45**(11), 95 (2021)
18. Ziaoddini, K.: Socially Aware Music Recommendation: A Multi-Modal Graph Neural Networks for Collaborative Music Consumption. *arXiv:2511.05497* (2025)